

CP3 复习 Mock Test

Date 955 958 2638.

一. 有关命令行的各种写法

argc	命令参数个数, 包括程序名本身 (等于 argv 里一共有多少个元素)		☆) 空格分割参数
与 argv 相关的表达式	类型	含义	引号内第一个空字符串 " " 也是一个
argv	char**	参数数组的指针, 指向 argv[0] 的指针	
argv[i]	char*	第 (i+1) 个参数字符串指针, 指向那个字符串第一个字符	
*argv[i]	char	第 (i+1) 个参数字符串的第一个字符	
argv[i]+n	char*	第 (i+1) 个参数字符串的第 (n+1) 个字符的指针	
*(argv[i]+n)	char	第 (i+1) 个参数字符串的第 (n+1) 个字符	
argv+i	char**	指向第 (i+1) 个参数的指针	
(argv+i)	char	第 (i+1) 个参数字符串指针 (即 argv[i])	
((argv+i)+n)	char	第 (i+1) 个参数字符串的第 (n+1) 个字符	
argv	char	第 1 个参数 (程序名)	
**argv	char	第 1 个参数的第一个字符	

核心规律

1. $argv[i] == *(argv+i)$ 千万别弄错了, $argv+i$ 是指向 $argv[i]$ 的指针

2. $*argv[i] == **argv+i$

3. $argv[i]+n == *(argv+i)+n$

4. $*argv[i]+n$ 这个不是指针+n 而是字符+n, 一定不要弄错了! 不要与 $*(argv[i]+n)$ 混淆

5. 字符串最后会自动补 \0, 千万别忘了!!!

二. 字对齐 (☆考试指针统一按 8 位处理)

① 同类型放在一起看, eg. `char lname[25]; char fname[15];` → +0

② 前面 byte 的总和必须是后面那个成员的数据类型长度的整数倍, 若不够需 padding

(注: `long int: 4`, `pointer: 8` for DEV(64位), `4` for (32位), `double: 8`)

③ 对于最后一个成员, 加完需满足 struct 的 length 是其所有成员的数据类型长度的最大值的整数倍, 若不够, 需 padding

eg: `typedef struct {`

`char lname[25];`

`int account;`

`char fname[15];`

`} node;`

$(25+3)+4+(15+1)=48$

`typedef struct this1 {`

`char lname[25];`

`char fname[15];`

`int account_no;`

`char bb[1];`

`struct this1 * this2;`

`{`

$40+4+(1+3)+8=56$

FALCON

三. 类型及其转换

1. 输出
- ① 输出 `%zu` 输出 `sizeof(int)` 的 `size_t` 类型, 通常是字节数 (bytes)
 - ② 整数
 - `%d` 有符号十进制 $\mathbb{Z}$
 - `%o` 八进制
 - `%u` 无符号 $\mathbb{Z}$
 - `%x` 十六进制 (小写)
 - `%X` 十六进制 (大写)
 - ③ 字符 (串)
 - `%c` 单个字符
 - `%s` 字符串
 - ④ 浮点数
 - `%f` float/double
 - `%lf` double
 - ⑤ 指针
 - `%p` 指针 eg. `%p, argv[0]` 第一个字符串的首字符地址
 - `%p, &argv[0]` 指针 `argv[0]` 自己的地址

2. 类型转换 (详情见 Tutorial 2 笔记)

① 算之前先统一类型

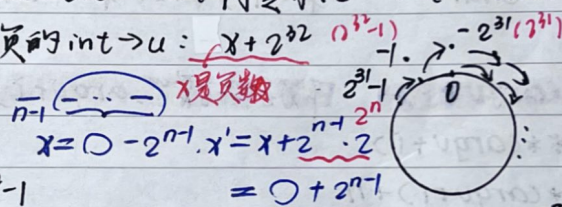
promotion: (i) more bytes (ii) unsigned (iii) float/double

- ② $\mathbb{Z} \div \mathbb{Z} = \mathbb{Z}$! 最后给 double 太迟了
- ③ char 运算时自动提升为 int
- ④ 赋值时才发生截断 double $\rightarrow$ int 向零取 $\mathbb{Z}$ int $\rightarrow$ char 只保留低位
- ⑤ int $\rightarrow$ unsigned int 真的 int $\rightarrow$ u: $x + 2^{32}$ (if $x < 0$)

• int 是 32 位

• signed $-2^{31} \sim 2^{31}-1$

• unsigned int $0 \sim 2^{32}-1$



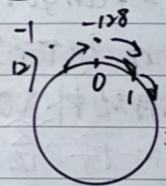
⑥ char 同理

$-128 \sim 127$

'char' c = 125; $\rightarrow$ 1 byte = 8 bits $-2^7 \sim 2^7-1$

do { printf("%d", c); while(c++);

执行 132 次



'\0' = 0 '\0' = 48 'A' = 65 'a' = 97 ' ' = 32 '\n' = 10

⑦ (详情见 PHASE 1 CHAP 1 笔记)

$(43)_{10} = (101011)_2$

2 | 43
2 | 21 ... 1
2 | 10 ... 1
2 | 5 ... 0
2 | 2 ... 1
2 | 1 ... 0
0 ... 1

LSB
MSB

$(0.3125)_{10} = (0.0101)_2$

$0.3125 \times 2 = 0.625$

$0.625 \times 2 = 1.25$

$0.25 \times 2 = 0.50$

$0.50 \times 2 = 1.00$

MSB
LSB

binary - octal / 16

$\mathbb{Z} \rightarrow$ 十六: 以小数点为分界, 整数部分向左每 4 位一组, 小数部分向右每 4 位一组, 最后不够 4 位的地方补 0

⑧ 二补码的负数 $\rightarrow$ 正数 $\rightarrow$ 记得加负号

四 * 相关的表示

① $*p++ = *(p++)$ $++$ 的优先级 $> *$ p 会指向下一个元素, 但解引用的是移动之前的 p ② $(*p)++$ p 指向的元素的值会 $+1$ ③ $*p++ = *$ $*p = *p++$ 改值不改指针

④ 匿名结构体定义变量

④ $*++p = *(++p)$ p 会指向下一个元素, 解引用的是移动之后的 p

struct {

struct st {

int a;

}

int a;

char b[5];

char b[5];

};

struct st s;

⑤ $++*p = ++(*p)$ p 指向的元素的值会 $+1$ ⑥ $*p+++ = *q$ p 原来指向的位置的值 $+*q$, 之后 p 再指向下一个元素

五 struct 的各种命名

a;

① struct st { int x; } a[10]; 定义了一个结构体类型叫 struct st, 同时又定义了一个变量名 a

② typedef struct st { int x; } node; 定义类型别名 node, 它代表 len 为 10 的 struct st 数组, 它代表指向 struct st 的指针类型

{ node[10]; } *node;

node a; // a 是 struct st 类型的 10 个元素的数组, 相当于 struct st a[10];

// 相当于 struct st *a;

六. 运算符优先级

① 后缀运算符 $() \rightarrow \cdot x++, x--$ 后置自增自减返回先前值② 单目运算符 $+ - ! \sim *(\text{解引用}) \&(\text{取地址}) ++x --x$ 前置返回之后值 右结合(右号) (逻辑非)
(负号) (按位取反) $++*p = ++(*p) \quad *++p = *(++p)$ ③ $* / \%$ ④ $+ -$ ⑤ $< >$ ⑥ 关系运算符 $< <= >= >$ ⑦ 相等运算符 $== !=$ ⑧ $\&\&$ ⑨ $\&$ ⑩ $!$ ⑪ $\&\&$ ⑫ $||$ ⑬ $?:$ ⑭ 赋值运算符 $= += \<\<= \&\&= \dots$

★

★

No.

七. 1000 8 1 1 1 1 f 0101 5 0110 6

Cn-2: 进入符号位的进位

Cn-1: 从符号位产生的进位

16个bit mask = $1 \ll 15$; n-1 0X8000

32 mask = $1 \ll 31$; 0X80000000

复1不是2!!!



八. 千万别写 for(int i=1; ...) ❌

九. 2^0 2^1 2^2 2^3 2^4 2^5 2^6 2^7 2^8 2^9

1024 512 256 128 64 32 16 8 4 2 1

十. 最开始 head = NULL, 不能直接 p = head.